

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Engineering (CSE)

SEMESTER FINAL EXAMINATION
 DURATION: 3 HOURS

SUMMER SEMESTER, 2023-2024
 FULL MARKS: 150

CSE 4403: Algorithms

Programmable calculators are not allowed. Do not write anything on the question paper.

Answer all 6 (six) questions. Figures in the right margin indicate full marks of questions with corresponding COs and POs in parentheses.

-
1. a) There are five problems with the following characteristics: 5 × 2
(CO1)
(PO1)
- Problem A can be solved and verified in polynomial time.
 - Problem B can be reduced to A in polynomial time and no reductions from NP-complete problems to B are known.
 - Problem C can be verified in polynomial time, but no polynomial-time algorithm is known to solve it. Problem A can be reduced to C in polynomial time.
 - Problem D is neither verifiable nor solvable in polynomial time, but all NP-complete problems can be reduced to D in polynomial time.
 - Problem E can be verified in polynomial time, and all NP-complete problems can be reduced to E in polynomial time.
- Classify each problem into its most likely complexity class (P, NP, NP-hard, NP-complete, or outside NP).
- b) Consider the following problems: 15
(CO2)
(PO2)
- Vertex Cover: Given a graph $G = (V, E)$ and an integer k , determine whether there exists a set of at most k vertices such that every edge in E has at least one endpoint in the set.
 - 3-SAT: Given a Boolean formula in CNF form with exactly three literals per clause, determine whether there exists an assignment to the variables that makes the formula true.
- Explain which direction of reduction is necessary to prove that Vertex Cover is NP-complete. Reduce the following 3-SAT problem to Vertex Cover problem:
- $$(x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_3 \vee x_4)$$
- Illustrate the corresponding graph and indicate the vertex cover that corresponds to a satisfying assignment.
2. a) You are given a set of activities, each with a start time and an end time. The goal is to select the maximum number of activities that do not overlap, i.e., no two selected activities should be running simultaneously. 7 + 6
(CO1)
(PO1)
- i. Prove that sorting the activities by their earliest finishing time and then selecting compatible activities greedily produces an optimal solution.
 - ii. Evaluate whether sorting the activities by their start time and then traversing in reverse order yields an optimal solution. Justify your answer with a proof or a counterexample.
- b) Apply Huffman Encoding to the string: **aaaabbccedddde**. Determine the Huffman Code for each character. Show all the steps of the encoding process, and include a diagram of the Huffman Tree. 12
(CO2)
(PO2)

3. a) For each of the following code snippets, determine the worst case time complexity using Big-O notation, assuming no code optimization is done by the compiler. Explain your answers briefly.

12
(CO1)
(PO1)

```
someFunction(int n, int m)
{
    for(i = 1; i < n; i *= 2)
    {
        for(j = m; j > 0; j /= 2)
        {
            // do something (with fixed no. of operations)
        }
    }
}
```

```
range(ara, n)
{
    mx = -inf, mn = inf;

    for(i = 0; i < n; i++) mx = max(mx, ara[i]);
    for(i = 0; i < n; i++) mn = min(mn, ara[i]);

    return mx - mn;
}
```

```
breakDance(n)
{
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            if(j > 5) break;
        }
    }
}
```

```
mixedSort(ara, n)
{
    if(n < 10) selectionSort(ara, n);
    else if(n < 100) bubbleSort(ara, n);
    else if(n < 1000) insertionSort(ara, n);
    else print("Sorry: ( Can not sort large array!");
}
```

Assume that the `selectionSort`, `bubbleSort`, and `insertionSort` functions are implemented according to their standard algorithmic definitions, without any modifications.

- b) A person wants to climb a staircase with n steps. At each move, the person can climb either 1 step or 2 steps. The task is to determine the total number of distinct ways to reach the top. Two solution methods are explained below:

Method 1: Let $f(i)$ denote the number of ways to reach step i . The recursion is defined as $f(i) = f(i-1) + f(i-2)$ with $f(0) = 1$ and $f(1) = 1$.

For $n = 5$, the recursive calls produce the following subproblems:

$$\begin{aligned} f(5) &= f(4) + f(3) \\ &= f(3) + f(2) + f(2) + f(1) \\ &= f(2) + f(1) + f(1) + f(0) + f(1) + f(0) + f(1) \\ &= f(1) + f(0) + f(1) + f(1) + f(0) + f(1) + f(0) + f(1) \end{aligned}$$

Method 2: Start from the base cases and compute the number of ways iteratively:

$$\begin{aligned} \text{ways}[0] &= 1, \quad \text{ways}[1] = 1 \\ \text{ways}[2] &= \text{ways}[1] + \text{ways}[0] = 2 \\ \text{ways}[3] &= \text{ways}[2] + \text{ways}[1] = 3 \\ \text{ways}[4] &= \text{ways}[3] + \text{ways}[2] = 5 \\ \text{ways}[5] &= \text{ways}[4] + \text{ways}[3] = 8 \end{aligned}$$

- i. Identify which problem-solving technique is used for each of the aforementioned methods, and suggest which one is the better between them. Justify your answer. 8
(CO1)
(PO1)
5
 - ii. Estimate the time and space complexity for both methods. (CO2)
(PO2)
4. a) Consider n cities are connected by directed roads. Each road is associated with a travel time and a transportation cost, where some roads may provide a monetary reward instead of charging a cost. The journey always starts at city 1 and ends at city n . During the journey, a vehicle change is mandatory at every intermediate city before proceeding further. For each of the following scenarios, propose an appropriate algorithmic approach and justify your answer. 4 × 3
(CO3)
(PO2)
- i. Find the route that minimizes the total cost, taking into account that some roads may provide rewards instead of charging for transport.
 - ii. Find the route that minimizes the total travel time.
 - iii. Determine the route that minimizes the number of vehicle switches.
- b) You are an Uber driver traveling through a network of 5 cities connected by one-way roads, where each road has an associated travel time. You are given an 5×5 matrix D , where $D[i][j]$ represents the minimum travel time from city i to city j (This may not represent a direct edge between the cities). Your journey always starts at city 1 and ends at city 5. 8
(CO)
(PO)

$$D = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 3 & 5 & 4 & 6 \\ 4 & 0 & 4 & 1 & 3 \\ 2 & 5 & 0 & 4 & 6 \\ 5 & 2 & 3 & 0 & 2 \\ 3 & 4 & 1 & 2 & 0 \end{bmatrix} \end{matrix}$$

There are k pickup requests for all cities, such that $1 < k < 5$. For each such request, it is necessary to determine whether you can visit city k and still complete the journey from city 1 to city 5 without increasing the minimum travel time. Propose a suitable solution and apply it to the given matrix D to compute the total number of valid pickup requests. Explain your answer briefly.

- c) Bogo Sort is a sorting algorithm that repeatedly shuffles the elements of a list randomly until the list happens to be sorted. Given its behavior, identify the type of randomized algorithm that Bogo Sort represents with a brief explanation. 5
(CO1)
(PO1)
5. a) Given an array of integers $x_1, x_2, \dots, x_n$, suppose there exists an index k ($1 < k < n$) such that: 13
(CO1)
(PO1)
- $x_1 < x_2 < \dots < x_k$ (strictly increasing), and
 - $x_k > x_{k+1} > \dots > x_n$ (strictly decreasing).
- Can this array be sorted in $O(n)$ time and space? If yes, describe the algorithm; if not, explain why.
- b) Consider the Code Snippet 1 and answer the following questions. 4 + 8
(CO2)
(PO2)
- i. Determine the time complexity of running mergelessSort for an array of size n .
 - ii. If $a = [1, 5, 4, 2, 7, 7, 3, 6]$, what will be the output of `mergelessSort(a, 0, 7)`? Explain your answer briefly.

```
void mergelessSort(int ara[], int l, int r)
{
    if(l == r) return;

    int m = (l + r) / 2;
    mergelessSort(ara, l, m);
    mergelessSort(ara, m + 1, r);
}
```

Code Snippet 1: A code snippet for Question 5.b

6. a) In the Bellman-Ford algorithm, all edges of a graph are relaxed repeatedly to compute the shortest distances from a source node. After performing this relaxation process for $n - 1$ iterations (where n is the number of vertices), the algorithm performs one final (the n -th) iteration to check for further updates. 10
(CO1)
(PO1)
- Prove that if any distance gets updated during this n -th iteration, the graph must contain a negative-weight cycle.
- b) We have four persons (p_1, p_2, p_3, p_4) and four jobs (j_1, j_2, j_3, j_4), with a compatibility matrix shown in Table 1 (1 = compatible, 0 = not). Each person can be assigned at most one job, and each job can be assigned to at most one person. The goal is to assign jobs so that the maximum number of persons receive a compatible job. 15
(CO3)
(PO2)
- Explain how this problem can be solved using Ford-Fulkerson algorithm and apply your solution to the given matrix to find the result.

Table 1: Compatibility matrix for Question 6.b.

	j_1	j_2	j_3	j_4
p_1	0	1	0	1
p_2	1	0	1	1
p_3	0	0	0	0
p_4	1	1	0	0